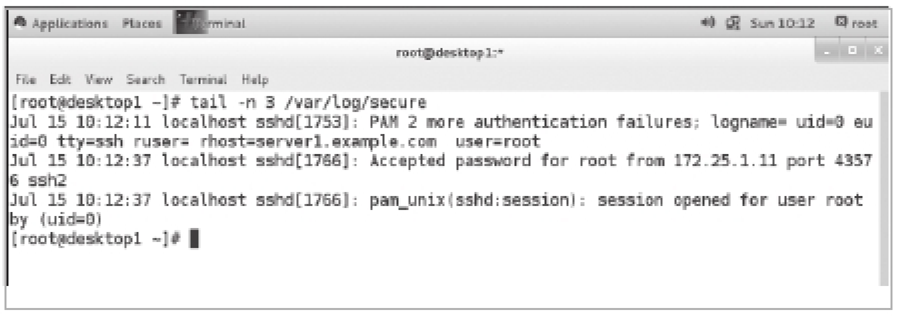


```
Applications  Places  Terminal                         Sun 10:12  root
root@desktop1:~
File  Edit  View  Search  Terminal  Help
[root@desktop1 ~]# tail -n 3 /var/log/secure
Jul 15 10:12:11 localhost sshd[1753]: PAM 2 more authentication failures; logname= uid=0 eu
id=0 tty=ssh ruser= rhost=server1.example.com  user=root
Jul 15 10:12:37 localhost sshd[1766]: Accepted password for root from 172.25.1.11 port 4357
6 ssh2
Jul 15 10:12:37 localhost sshd[1766]: pam_unix(sshd:session): session opened for user root
by (uid=0)
[root@desktop1 ~]#
```

Figure 2: Authentication logs are verified

Send a syslog message with a *logger*

The *logger* command can send messages to the *rsyslog* service. By default, it sends the message to the facility user with a severity notice (*user.notice*) unless specified otherwise, with the *–p* option. It is especially useful to test changes to the *rsyslog* configuration.

To send a message to *rsyslogd* that gets recorded in the */var/log/boot.log* file, execute the following command:

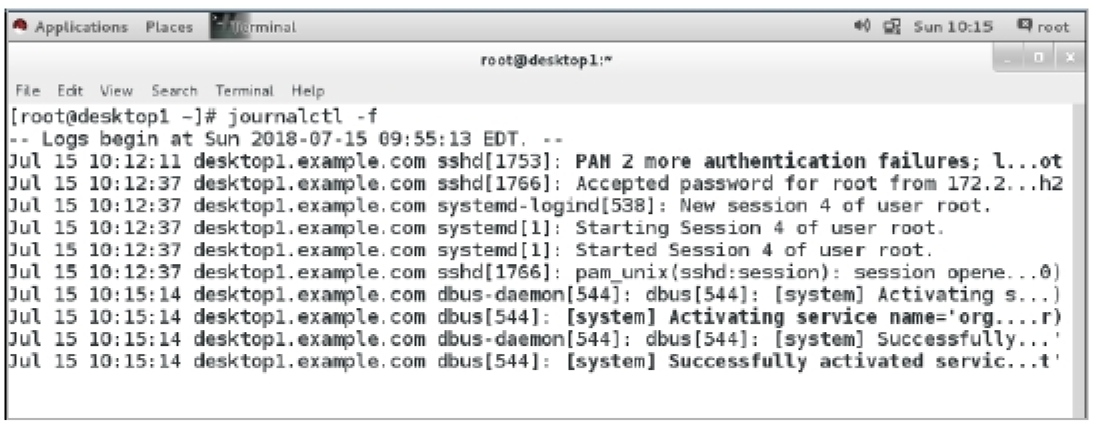
```
# logger -p local 7.notice "Log entry created on test7"
```

Reviewing system journal entries (finding events with *journalctl)*

The *systemd* journal stores logging data in a structured, indexed binary file. This data includes extra information about the log event. For syslog events, this can include the facility and priority of the original message.

```
[root@desktop1 ~]# journalctl
-- Logs begin at Sun 2018-07-15 09:55:13 EDT, end at Sun 2018-07-15 10:12:37 EDT. --
Jul 15 09:55:13 localhost systemd-journal[89]: Runtime journal is using 8.0M (max 92.0M, le
Jul 15 09:55:13 localhost systemd-journal[89]: Runtime journal is using 8.0M (max 92.0M, le
Jul 15 09:55:13 localhost kernel: Initializing cgroup subsys cpuset
Jul 15 09:55:13 localhost kernel: Initializing cgroup subsys cpu
Jul 15 09:55:13 localhost kernel: Initializing cgroup subsys cpuacct
Jul 15 09:55:13 localhost kernel: Linux version 3.10.0-123.el7.x86_64 (mockbuild@x86-017.bu
Jul 15 09:55:13 localhost kernel: Command line: BOOT_IMAGE=/boot/vmlinuz-3.10.0-123.el7.x86
Jul 15 09:55:13 localhost kernel: [Firmware Bug]: TSC doesn't count with P0 frequency!
Jul 15 09:55:13 localhost kernel: e820: BIOS-provided physical RAM map:
Jul 15 09:55:13 localhost kernel: BIOS-e820: [mem 0x0000000000000000-0x000000000009fbff] us
Jul 15 09:55:13 localhost kernel: BIOS-e820: [mem 0x000000000009fc00-0x000000000009ffff] re
Jul 15 09:55:13 localhost kernel: BIOS-e820: [mem 0x00000000000f0000-0x00000000000fffff] re
Jul 15 09:55:13 localhost kernel: BIOS-e820: [mem 0x0000000000100000-0x000000007ffdbfff] us
Jul 15 09:55:13 localhost kernel: BIOS-e820: [mem 0x000000007ffdc000-0x000000007fffffff] re
Jul 15 09:55:13 localhost kernel: BIOS-e820: [mem 0x00000000fffc0000-0x00000000ffffffff] re
Jul 15 09:55:13 localhost kernel: NX (Execute Disable) protection: active
Jul 15 09:55:13 localhost kernel: SMBIOS 2.8 present.
Jul 15 09:55:13 localhost kernel: DMI: Bochs Bochs, BIOS Bochs
Jul 15 09:55:13 localhost kernel: Hypervisor detected: KVM
Jul 15 09:55:13 localhost kernel: e820: update [mem 0x00000000-0x00000fff] usable ==> reser
Jul 15 09:55:13 localhost kernel: e820: remove [mem 0x000a0000-0x000fffff] usable
Jul 15 09:55:13 localhost kernel: No AGP bridge found
Jul 15 09:55:13 localhost kernel: e820: last_pfn = 0x7ffdc max_arch_pfn = 0x400000000
Jul 15 09:55:13 localhost kernel: MTRR default type: write-back
Jul 15 09:55:13 localhost kernel: MTRR fixed ranges enabled:
Jul 15 09:55:13 localhost kernel:   00000-9FFFF write-back
```

Figure 3: Output of the *journalctl* command



```
Applications  Places  Terminal                         Sun 10:15  root
root@desktop1:~
File  Edit  View  Search  Terminal  Help
[root@desktop1 ~]# journalctl -f
-- Logs begin at Sun 2018-07-15 09:55:13 EDT. --
Jul 15 10:12:11 desktop1.example.com sshd[1753]: PAM 2 more authentication failures; l...ot
Jul 15 10:12:37 desktop1.example.com sshd[1766]: Accepted password for root from 172.2...h2
Jul 15 10:12:37 desktop1.example.com systemd-logind[538]: New session 4 of user root.
Jul 15 10:12:37 desktop1.example.com systemd[1]: Starting Session 4 of user root.
Jul 15 10:12:37 desktop1.example.com systemd[1]: Started Session 4 of user root.
Jul 15 10:12:37 desktop1.example.com sshd[1766]: pam_unix(sshd:session): session opene...0)
Jul 15 10:15:14 desktop1.example.com dbus-daemon[544]: dbus[544]: [system] Activating s...)
Jul 15 10:15:14 desktop1.example.com dbus[544]: [system] Activating service name='org....r)
Jul 15 10:15:14 desktop1.example.com dbus-daemon[544]: dbus[544]: [system] Successfully...'
Jul 15 10:15:14 desktop1.example.com dbus[544]: [system] Successfully activated servic...t'
```

Figure 4: Output of *journalctl –f*

Important: In Red Hat Enterprise Linux 7, the *systemd* journal is stored in */run/log*, by default, and its contents are cleared after reboot. The setting can be changed by the systems administrator.

The *journalctl* command shows the full system journal, starting from the oldest log entry, when run as the root user. The output is shown in Figure 3.

The *journalctl* command highlights in bold text the messages of priority notice or warning; messages of priority error and higher are highlighted in red.

The key to successfully using the journal for troubleshooting and auditing is to limit the journal searches to only show relevant output. By default, *journalctl –n* shows the last ten log entries. It takes an optional parameter for how many of the last log entries should be displayed. To display the last five log entries, run the following command:

```
# journalctl -n 5.
```

When troubleshooting problems, it is useful to filter the output of the journal on the basis of the priority of journal entries. *journalctl –p* takes either the name or the number of the known priority levels, and shows the given levels and all higher level entries. The priority levels known to *journalctl* are *debug, info, notice, warning, err, crit, alert* and *emerg.*

To filter the output of the *journalctl* command to only list any log entry of priority *err* or above, run the following command:

```
# journalctl -p err
```

Similar to the *tail –f* command, *journalctl –f* outputs the last ten lines of the journal and continues to output new journal entries as they get written to the journal.

When looking for specific events, it is useful to limit the output to a specific time frame. The *journalctl* command has two options to limit the output to a specific time range, the *--since* and *--until* options. Both options take a time parameter in the format *YYYY-MM-DD hh:mm:ss.* If the date is omitted, the command assumes the current date, and if the time is omitted, the whole day, starting at 00:00:00, is assumed. Both options take yesterday, today and tomorrow as valid parameters in addition to the date and time field.

The output of all journal entries recorded today can be shown using the following command: